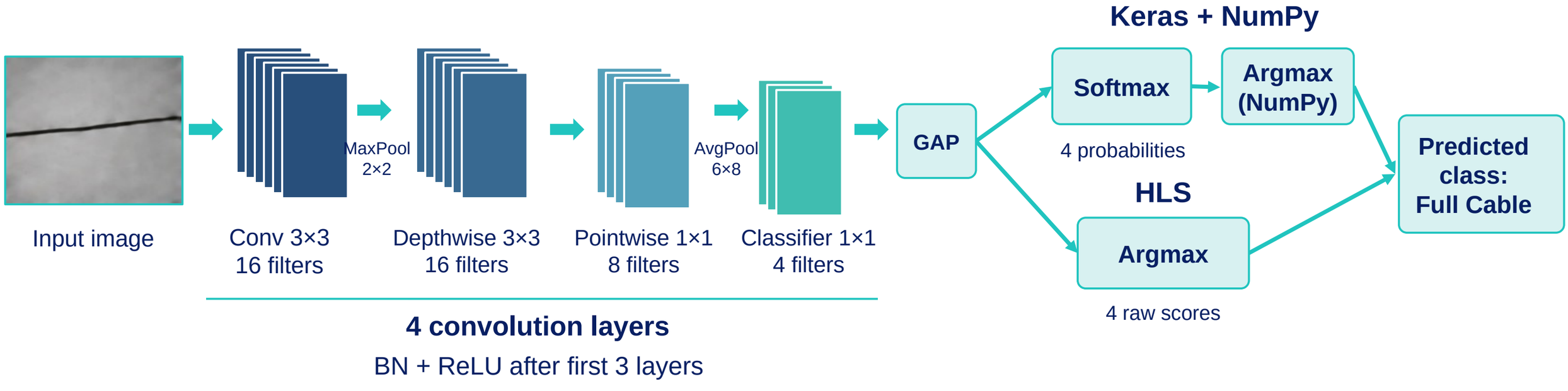
Lightweight CNNs for Embedded Vision

From convolution to an efficient image classifier



Dinesh Baniya Kshatri

PhD Student

Digital and High-Frequency Electronics Section

20 October 2026

Learning Objectives

By the end of this lecture, you will be able to:

01	Explain convolution and image feature extraction.
02	Compare standard and depthwise separable convolution.
03	Augment and split images by original, keeping validation and test unaugmented.
04	Preprocess images: resize to 60 × 80, convert to grayscale, and normalize.
05	Train, validate, and evaluate the CNN on the test set in Keras.
06	Interpret training and validation loss curves and confusion matrices.
07	Export trained model parameters for FPGA implementation.
08	Implement and verify the accelerator in Vitis HLS; synthesize to RTL.
09	Integrate the accelerator and implement the design in Vivado.
10	Generate the bitstream, program the FPGA, and verify its predictions.

How Convolution Extracts Image Features

A filter examines a small region and produces one value at each position.

Input image patch

1	1	1	1	1
1	1	1	1	1
0	0	0	0	0
1	1	1	1	1
1	1	1	1	1

Dark row represents the cable

0 = dark, 1 = bright

Example 3 × 3 filter

1	1	1
-2	-2	-2
1	1	1

Multiply corresponding values, then add

For the centered cable patch:

3 + 0 + 3 = 6

Feature map

-3	-3	-3
6	6	6
-3	-3	-3

Strong response along the cable

Stride 1, no padding

Our first convolution layer learns 16 filters, producing 16 feature maps.

Illustrative filter and pixels. Actual CNN filter weights are learned during training.

Filter Window and Feature Map

Move the filter by one pixel and reveal the next output value.

Image patch + filter

1	1	1	1	1
1	1	1	1	1
0	0	0×1	0×1	0×1
1	1	1×-2	1×-2	1×-2
1	1	1×1	1×1	1×1

Position 9 of 9

Multiply, then sum
Output = -3

Output feature map

-3	-3	-3
6	6	6
-3	-3	-3

Teal cell = current output

Example: stride 1, no padding. Custom CNN uses stride 1 with same padding.

Filter weights stay fixed. Output values form the feature map.

Convolution Output Size

Padding is on all four sides. Calculate height and width separately.

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

n_{in}	Input height or width
n_{out}	Output height or width
k	Convolution kernel size
p	Padding per side (all four sides)
s	Stride size

Why these operations?

+ 2p: top + bottom OR left + right.

− k: distance the window can move.

÷ s, then floor: number of full steps.

+ 1: include the starting position.

First layer: k = 3, p = 1, s = 1

Height: floor((60 + 2 − 3) / 1) + 1 = 60

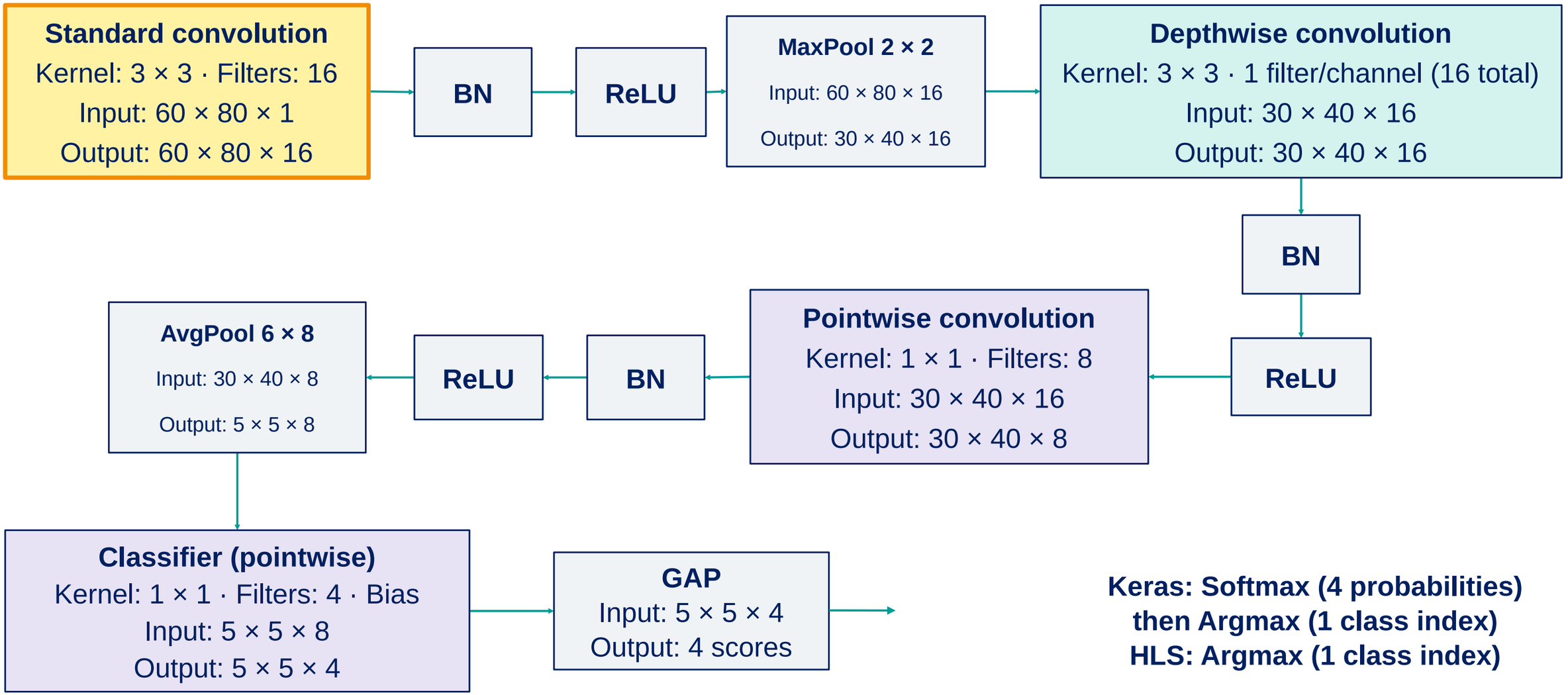
Width: floor((80 + 2 − 3) / 1) + 1 = 80

Padded size: (H + 2p) × (W + 2p). Output: 60 × 80 × 16.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Standard Convolution

A spatial filter extracts features from the grayscale image.

A 3 × 3 filter has 9 learned weights. Multiply pixels by weights and sum.

The same weights slide across the image to build one feature map.

16 filters learn 16 sets of weights. The filter count is a design choice.

One spatial patch				× learned filter weights		
1	1	1	×	1	1	1
0	0	0		-2	-2	-2
1	1	1		1	1	1

Multiply matching entries, then sum: 3 + 0 + 3 = 6

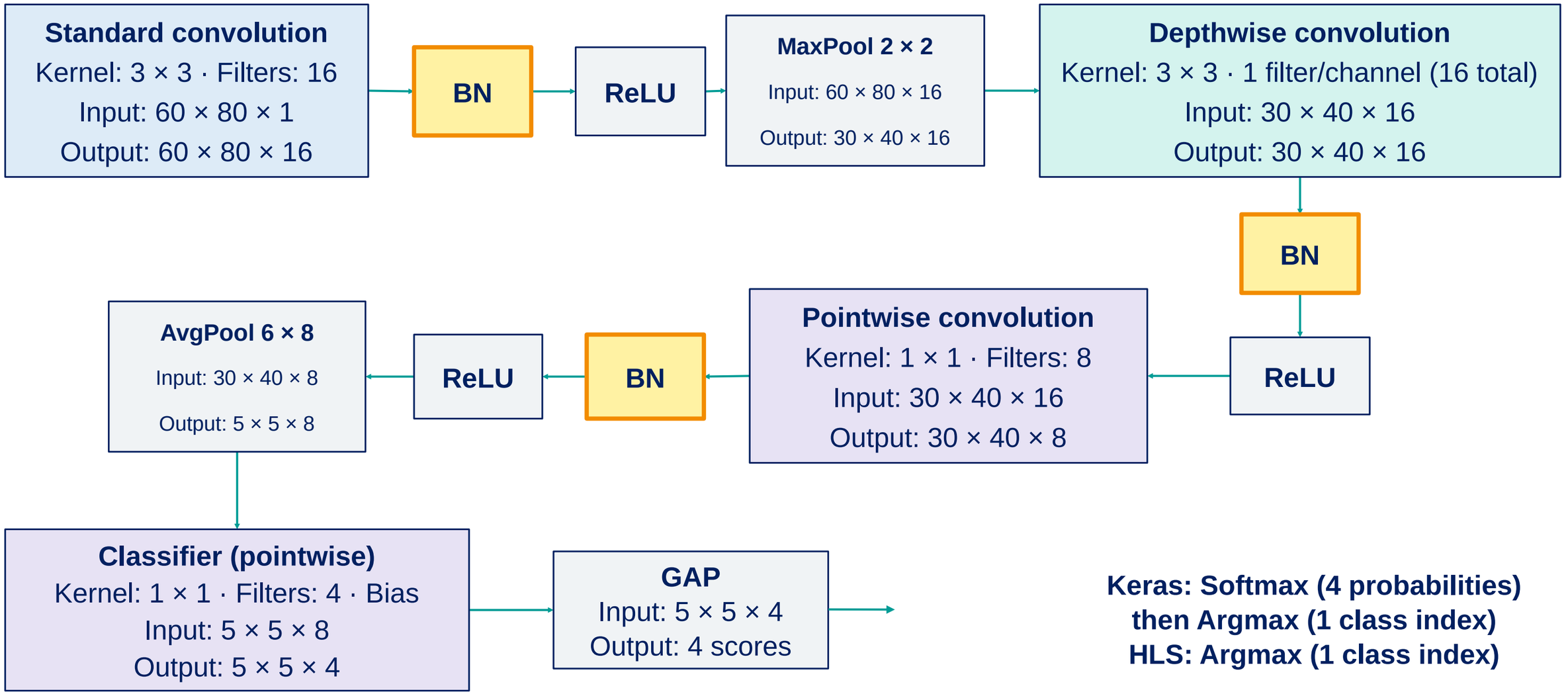
Illustrative weights. Actual weights are learned.

Input: 60 × 80 × 1. Output: 60 × 80 × 16. Stride 1, no bias.
Same padding: one zero border on all four sides preserves height and width.

Custom CNN: Complete Layer Sequence

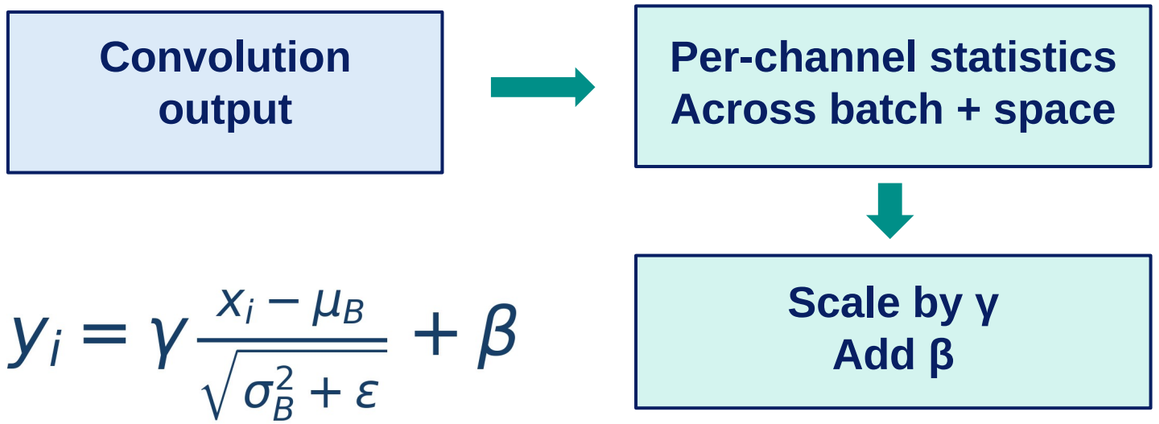
Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Batch Normalization



Normalize, then learn a scale and offset for each channel.

Why it helps: BN often makes training more stable and allows larger learning rates.

$$y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta$$

Symbol	Meaning
x_i	Input activation
μ_B	Mean of the activations in the batch
σ^2_B	Variance of the activations in the batch
ε	Small constant for numerical stability
γ	Learned scaling parameter
β	Learned shifting parameter
y_i	Output after batch normalization

For each channel, use convolution outputs from all B images and all H × W positions.

Mean: average those B × H × W values.
Variance: average their squared deviations from the mean.

Computes a separate mean and variance for each of the 16 channels, using N = B × 60 × 80 values per channel.

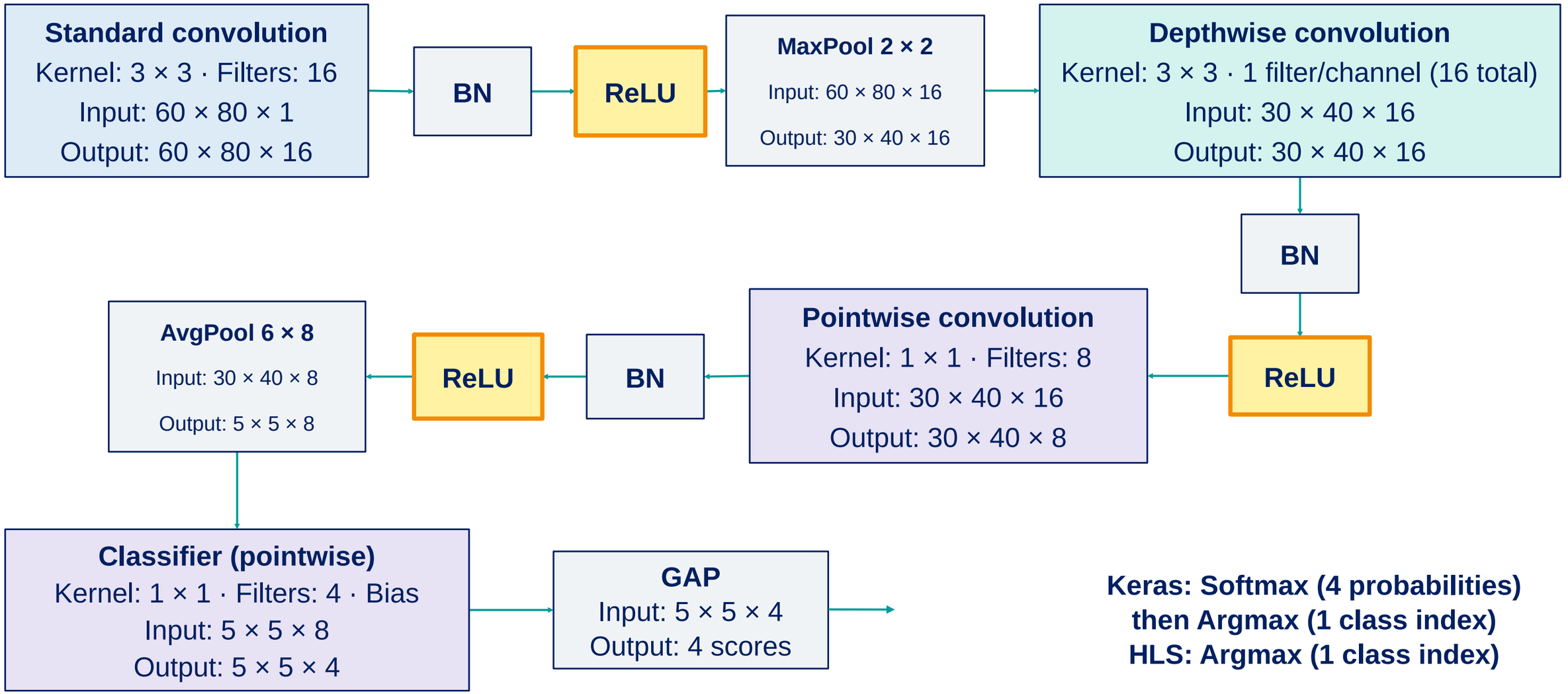
During inference, uses the stored moving mean and variance.

Batch normalization preserves the feature-map dimensions.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

ReLU Activation

Keep positive values and replace negative values with zero.

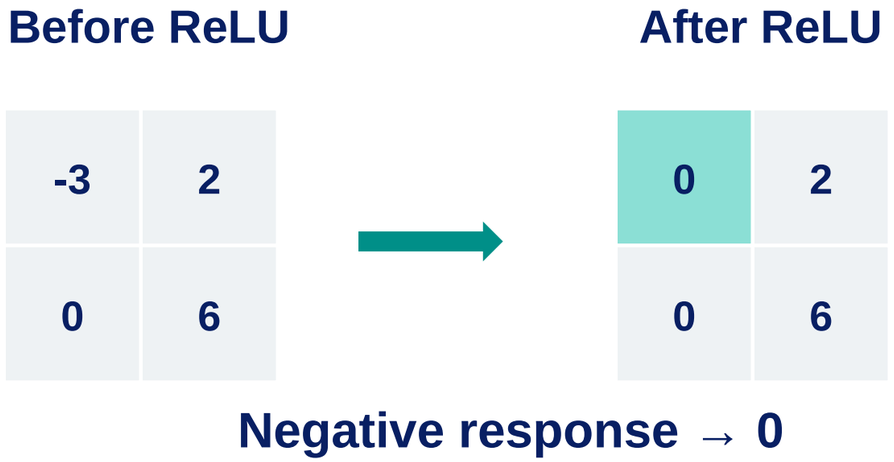
ReLU introduces a nonlinear transformation.

Nonlinearity lets the CNN learn complex patterns

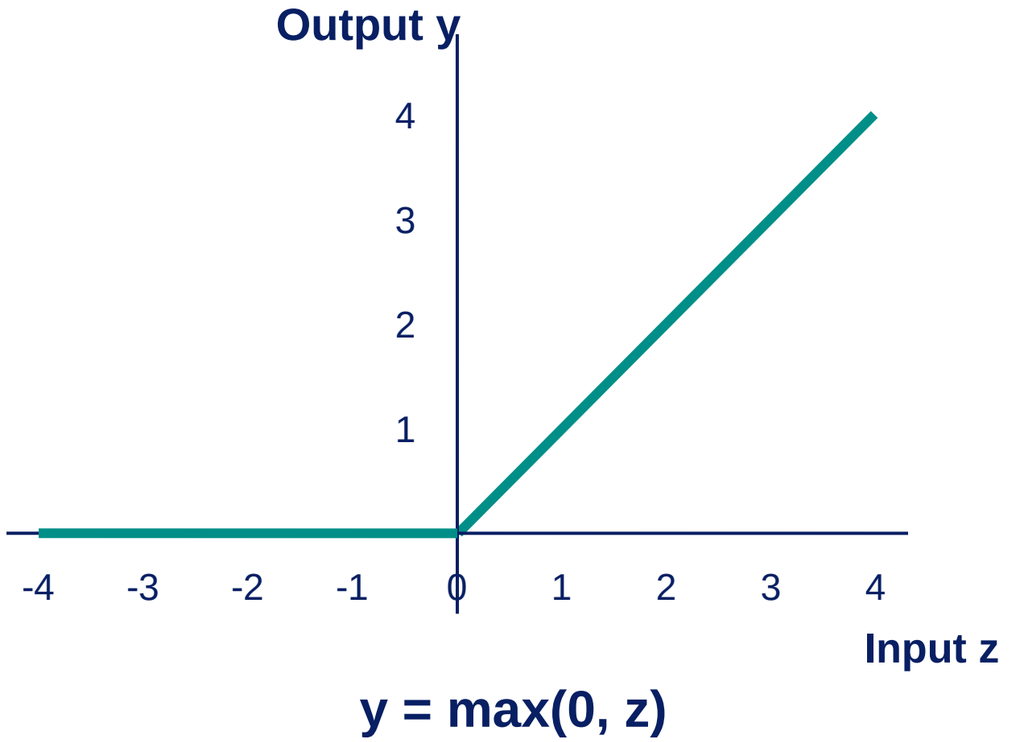
Apply y = max(0, z) independently to every feature value.

The input and output have the same dimensions.

Custom CNN: ReLU follows each of the first three BN layers.



Negative response → 0

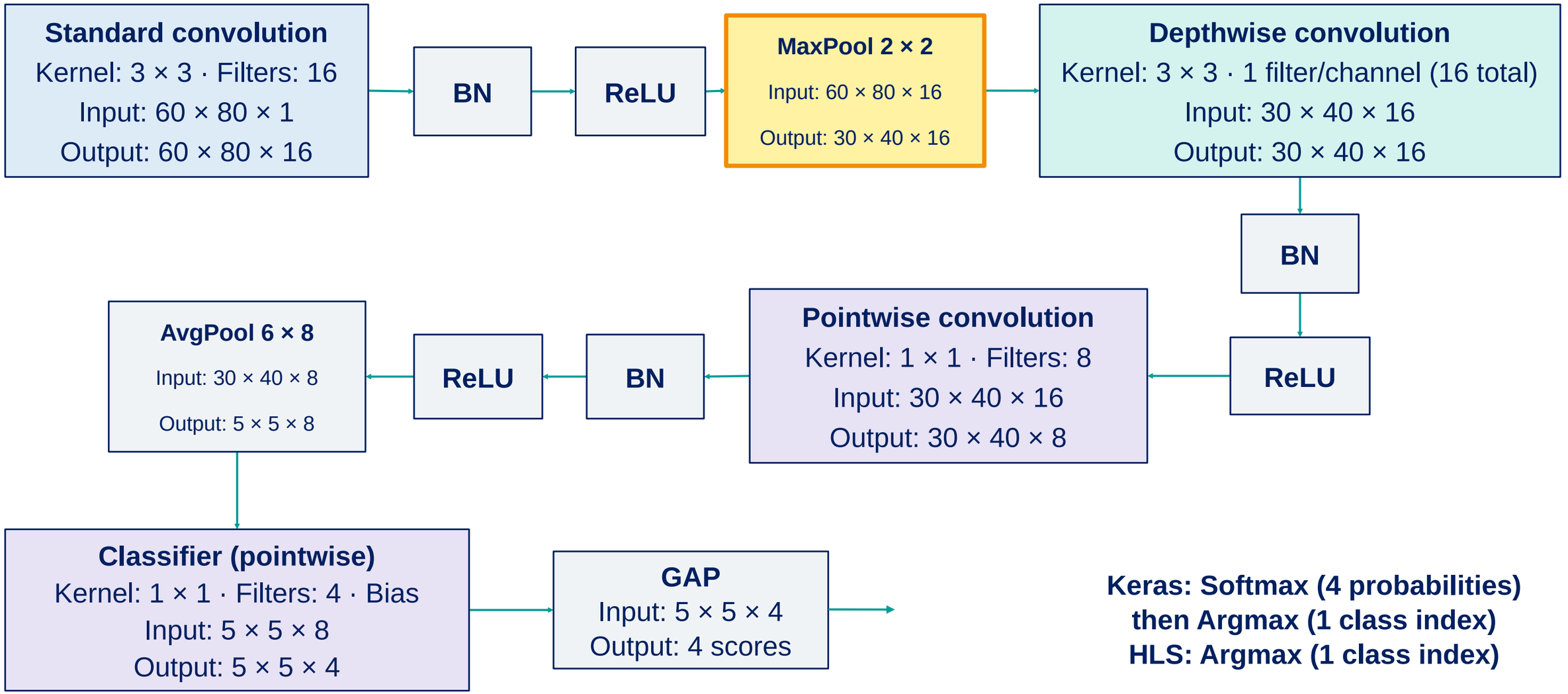


y = max(0, z)

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

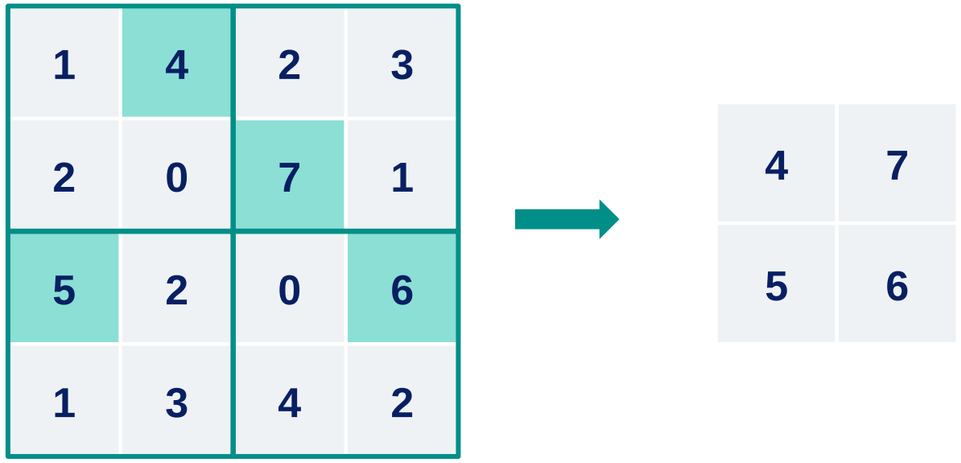
Max Pooling

Retain the strongest response in each local region.

Take the maximum value in each 2 × 2 window.

Move the window by 2 pixels. The windows do not overlap.

Height and width halve. Channels remain unchanged.



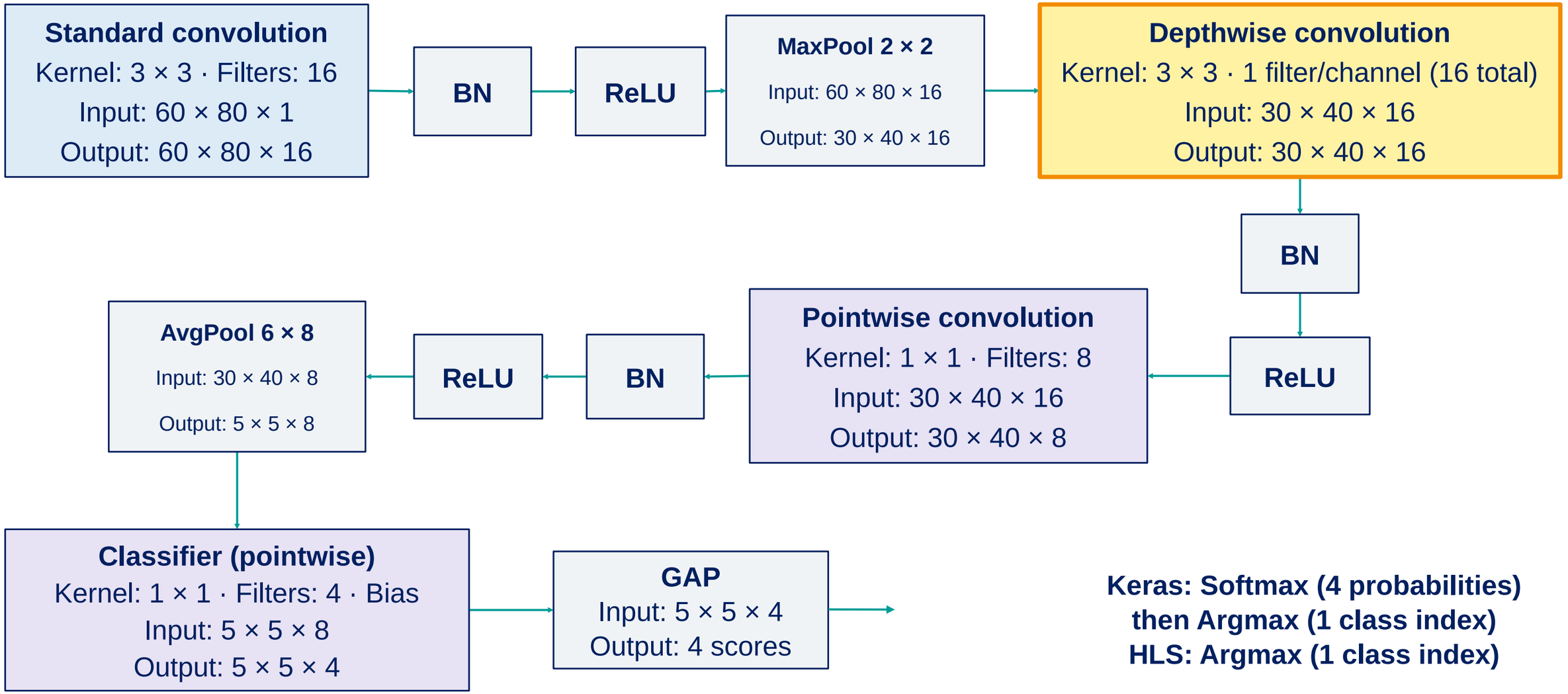
Each 2 × 2 block contributes its maximum.

Custom CNN: 60 × 80 × 16 input; 30 × 40 × 16 output. Pool size 2 × 2, stride 2.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Depthwise Convolution

Filter each feature channel separately.

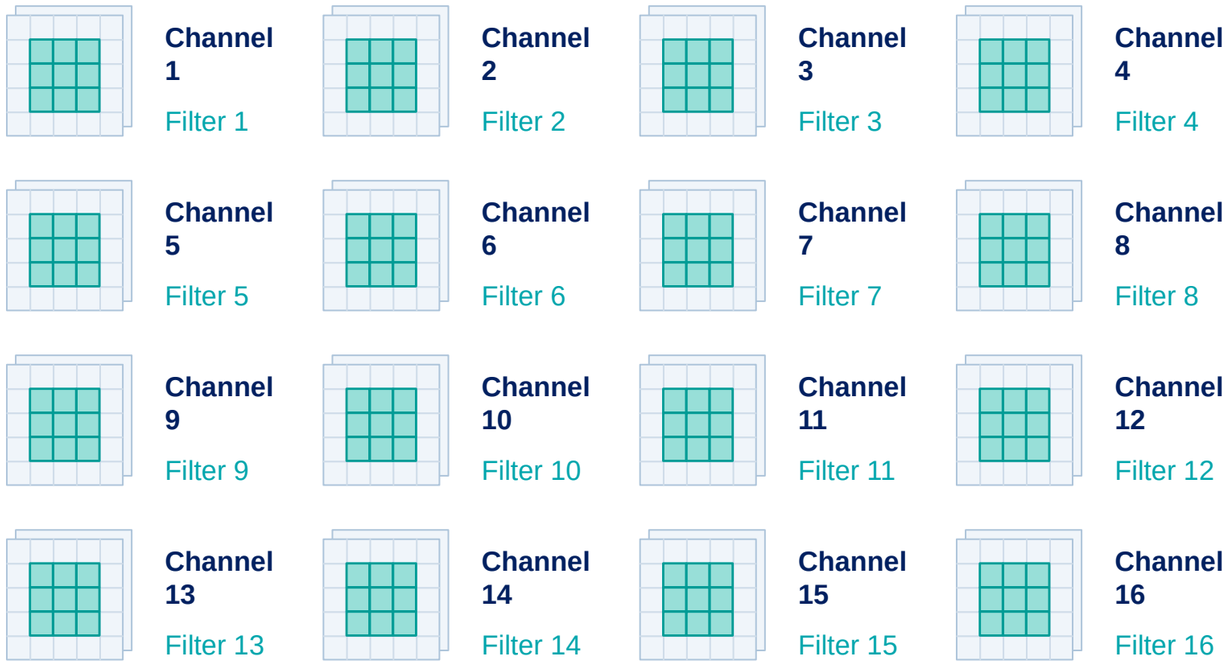
Each of the 16 input channels has its own 3 × 3 filter.

The filter uses values from its own channel. Channels do not mix.

16 separate output maps stack into a 30 × 40 × 16 tensor. They are not summed together.

- Weights: depthwise 144 (3 × 3 × 16) vs. standard 2,304 (3 × 3 × 16 × 16). Comparison: 16 input and 16 output channels.
- MACs: 172,800 (30 × 40 × 144) vs. 2,764,800 (30 × 40 × 2,304). 16× fewer (2,764,800 ÷ 172,800).

16 channels, each with its own 3 × 3 filter



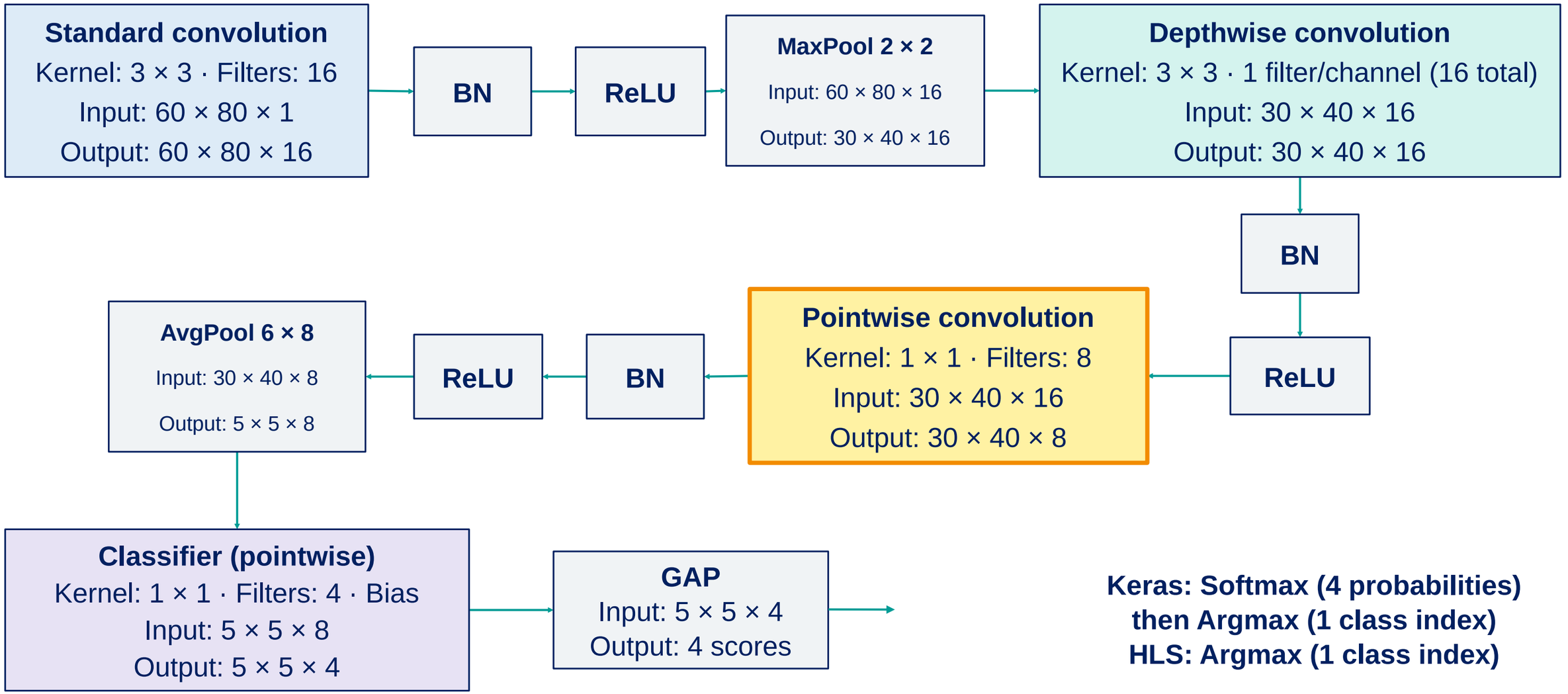
Each teal 3 × 3 patch uses a different learned filter.

Input and output: 30 × 40 × 16. Stride 1, no bias.
Same padding: one zero border on all four sides preserves height and width.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Pointwise Convolution

16 channel values at the same (i, j) → **Weighted sum for output channel r**

Mix channels at one spatial position.

Pointwise convolution follows depthwise convolution to combine information across channels.

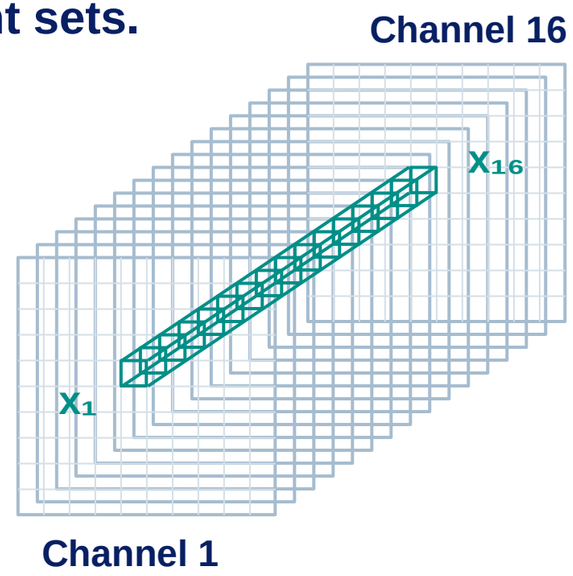
$$y_r(i,j) = \sum_{c=1}^{16} w_{c,r} X[i,j,c]$$

A 1 × 1 filter spans all 16 input channels at position (i, j).

Repeat with 8 different weight sets.

Multiply the 16 channel values by 16 weights and add them.

Symbol	Meaning
X	Entire input tensor: 30 × 40 × 16
i, j	Row and column of one spatial position
c	Input channel index: 1 to 16
r	Output channel index: 1 to 8
X[i, j, c]	One input value at (i, j), in channel c
w(c, r)	Learned weight from input c to output r
y_r(i, j)	One output value at (i, j), in channel r



8 filters produce 8 output feature maps.

16 teal cells = 16 input values

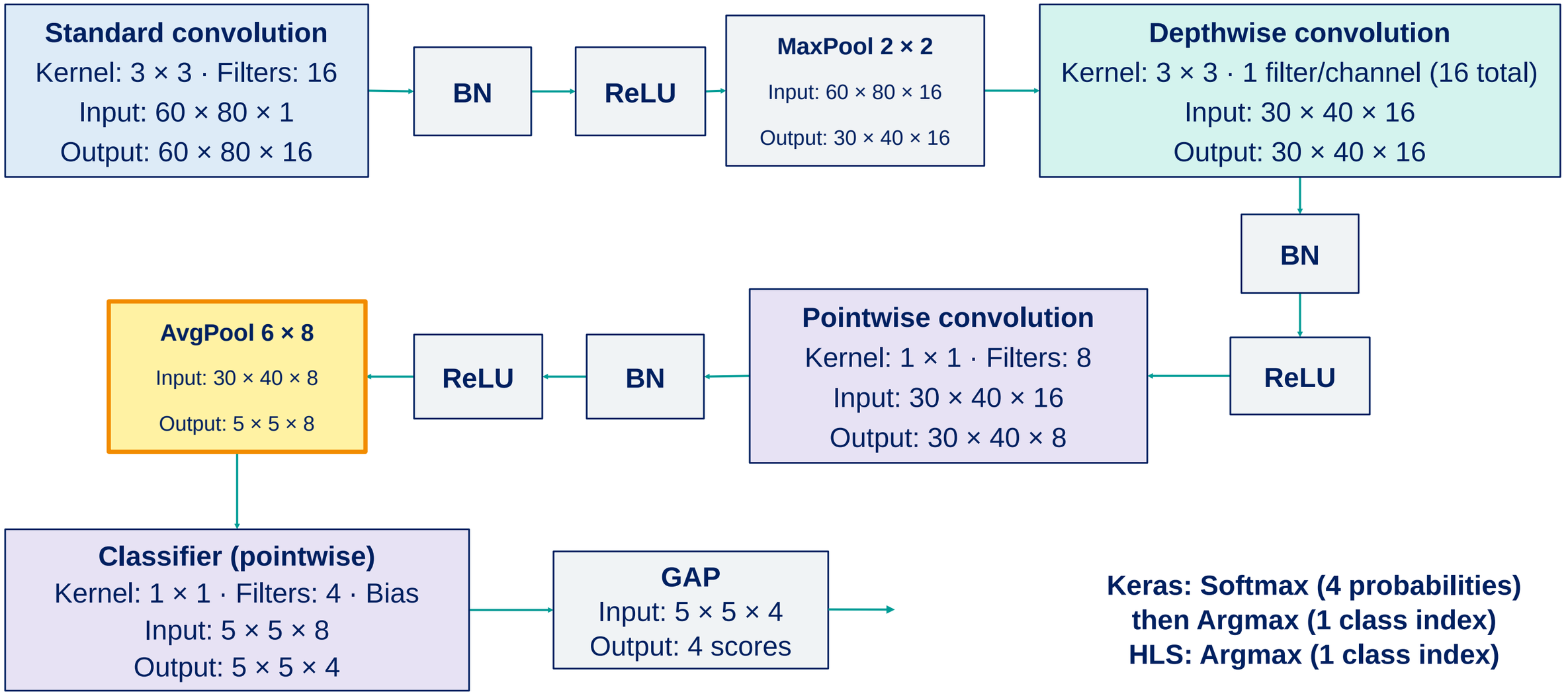
8 filters produce 8 spatial size stays unchanged.

Input: 30 × 40 × 16. Output: 30 × 40 × 8. No bias, then BN and ReLU.
Same padding with a 1 × 1 kernel needs no zero border. Height and width stay unchanged.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

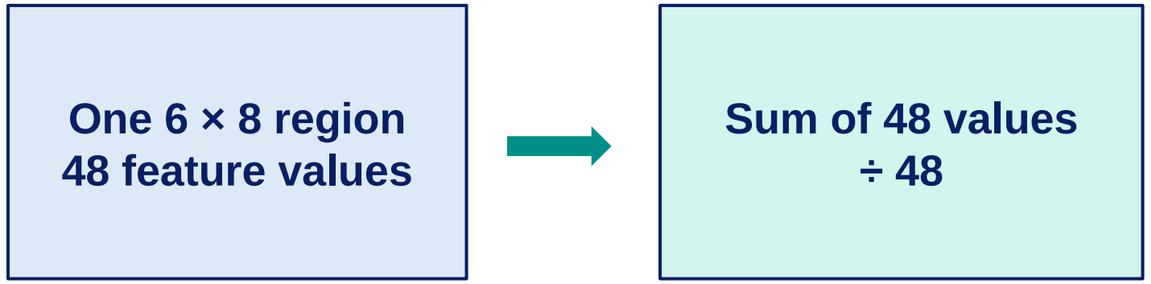
Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Average Pooling

Summarize each region using its mean.



Mean becomes one output value.

Average all 48 values in a 6 × 8 window, independently per channel.

Stride (6, 8) forms non-overlapping regions.

30 / 6 = 5 rows; 40 / 8 = 5 columns. All 8 channels remain.

One 6 × 8 feature region

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48

48 feature values

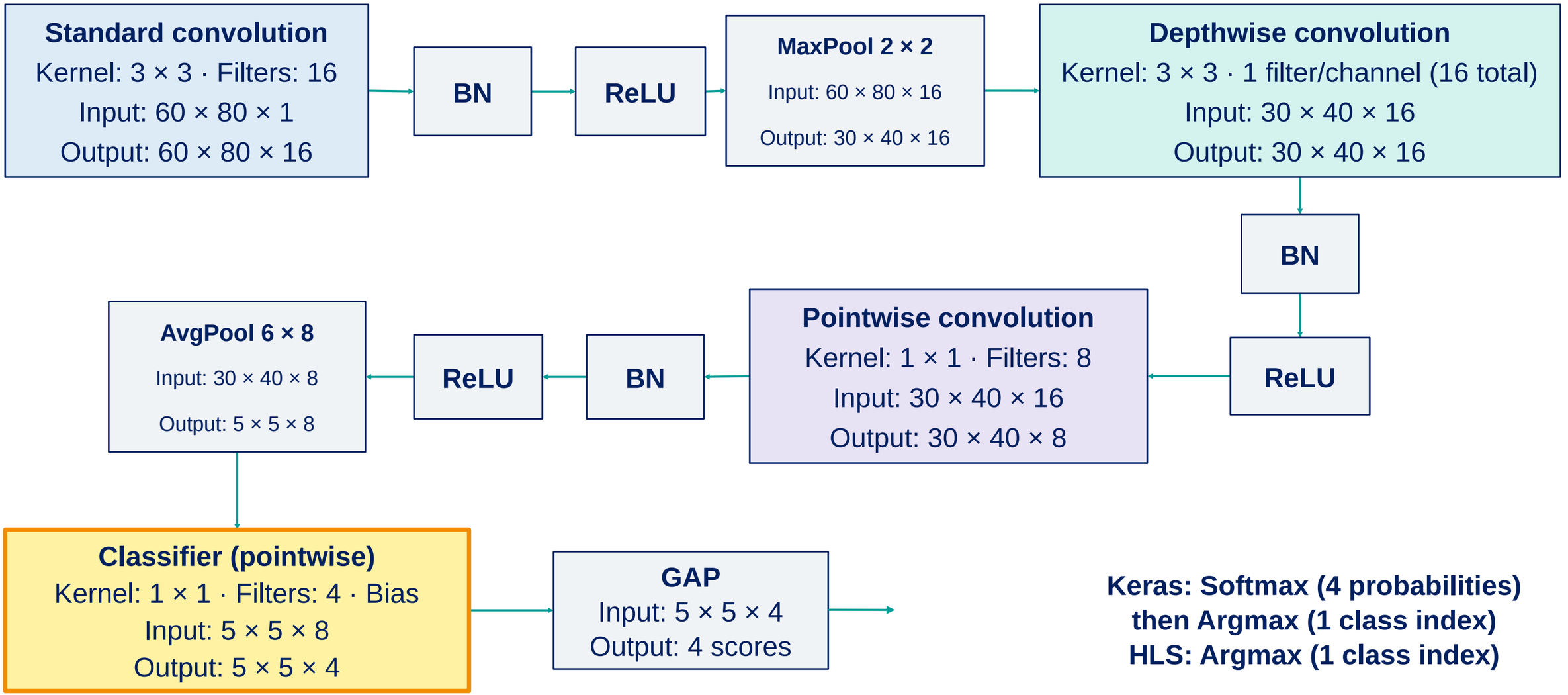
Example: sum = 1176, so average = 24.5

Custom CNN: 30 × 40 × 8 input; 5 × 5 × 8 output. No padding.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

Classifier Convolution

A 1 × 1 filter combines the 8 input feature channels to produce one score for each class.

$$y_{i,j,c} = \sum_{r=1}^{8} w_{r,c} \cdot h_{i,j,r} + b_c$$

Expanded: $y_{i,j,c} = w_{1,c} \cdot h_{i,j,1} + w_{2,c} \cdot h_{i,j,2} + \cdots + w_{8,c} \cdot h_{i,j,8} + b_c$

8 feature channels at position (i, j) → **4 weighted sums + 4 class biases**

Indices

i, j	Spatial position in the 5 × 5 feature map
r	Input channel index, from 1 to 8
c	Class index, from 0 to 3

Values and parameters

$h_{i,j,r}$	Input activation from channel r at (i, j)
$w_{r,c}$	Learned weight for channel r and class c
b_c	Bias for class c
$y_{i,j,c}$	Raw class score (logit) at (i, j)

At each of the 25 positions, the 4 filters produce 4 class scores.

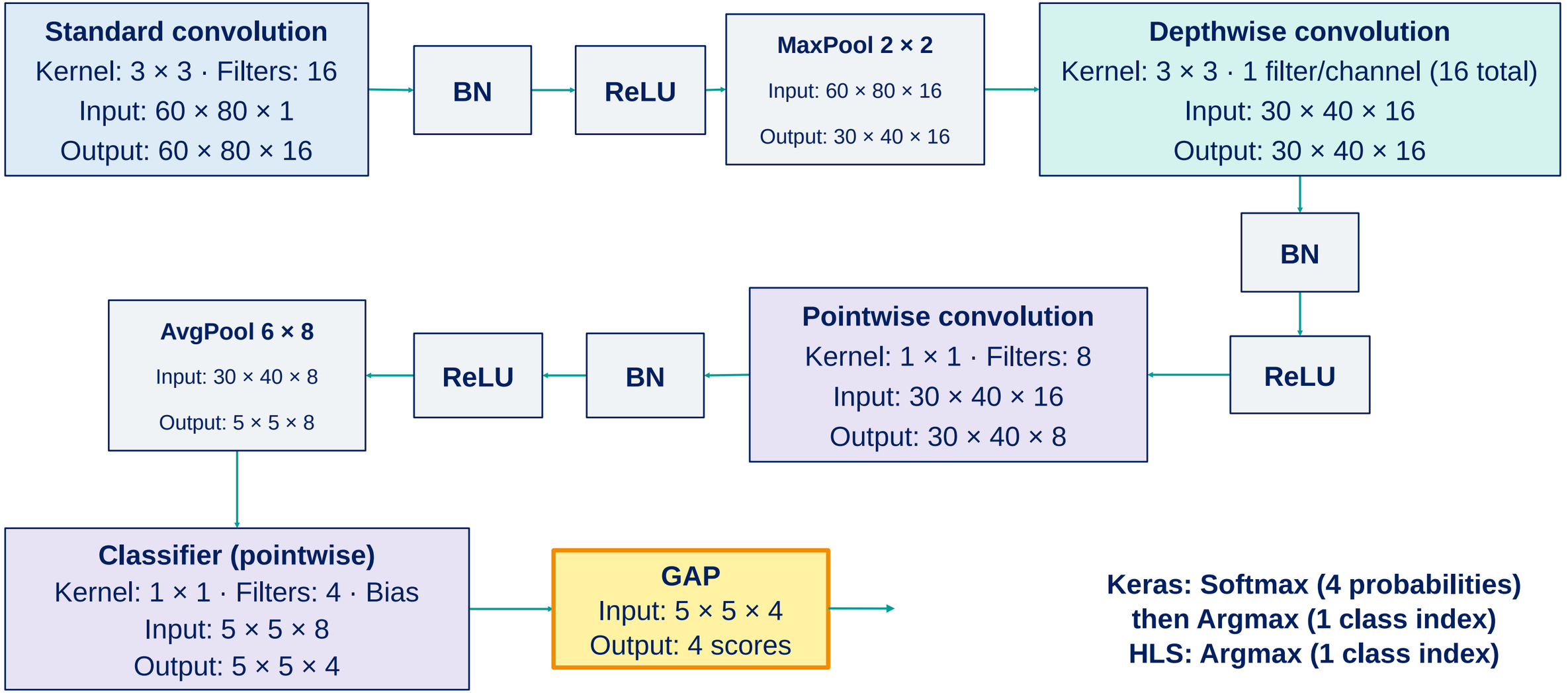
Input: 5 × 5 × 8 Kernel: 1 × 1 Output: 5 × 5 × 4

The outputs are raw class scores (logits). No batch normalization or ReLU follows.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



Keras: Softmax (4 probabilities)
then Argmax (1 class index)
HLS: Argmax (1 class index)

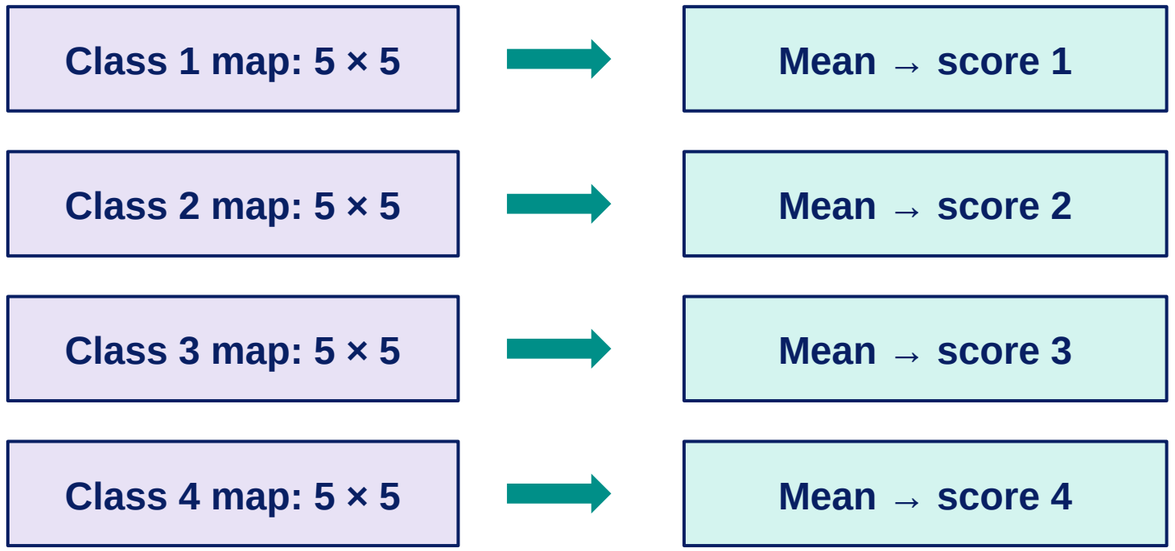
Global Average Pooling

Reduce each class-score map to one score.

Average every value in each 5 × 5 class map.

Each class contributes one score: sum of 25 values divided by 25.

Four class maps become a vector of four scores.



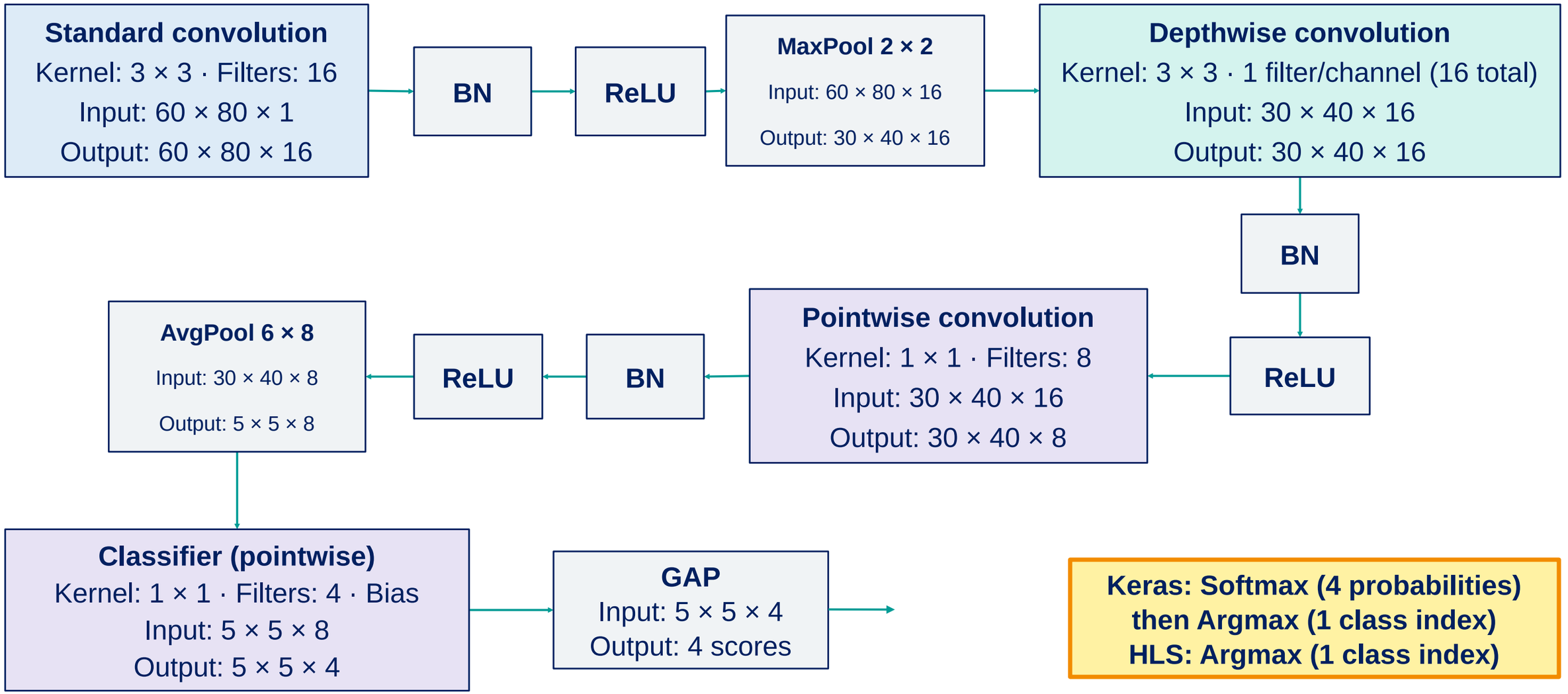
Average spatial positions, not classes.

Custom CNN: 5 × 5 × 4 input; 4 class scores output.

Custom CNN: Complete Layer Sequence

Grayscale input: 60 × 80 × 1

Tensor size: H × W × C = height × width × channels



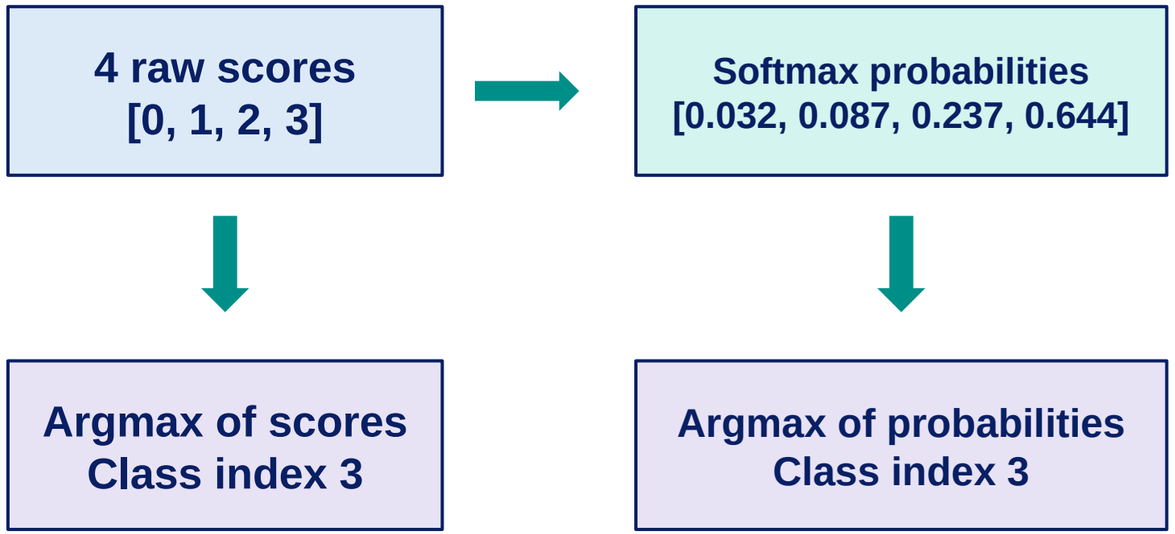
Softmax and Argmax

Probabilities and a class label are different outputs.

Training: softmax converts scores to probabilities for cross-entropy loss.

It is differentiable, so the loss provides gradients to update the weights. Argmax cannot do this.

Inference: argmax selects the largest score. Softmax keeps the same winner, so HLS can skip it.



Both select the same class. Values shown are illustrative.

Custom CNN: Keras produces probabilities. NumPy Argmax gives the label; HLS uses Argmax on scores.

CNN Architecture and Parameter Calculations

Shapes: height × width × channels, excluding the batch dimension

Layer	Input shape	Output shape	Kernel weights	Biases	Trainable parameters	Non-trainable parameters
Conv2D, 3 × 3	60 × 80 × 1	60 × 80 × 16	3×3×1×16 = 144	0	144	0
Batch normalization	60 × 80 × 16	60 × 80 × 16	0	0	2×16 = 32	2×16 = 32
ReLU	60 × 80 × 16	60 × 80 × 16	0	0	0	0
Max pooling, 2 × 2	60 × 80 × 16	30 × 40 × 16	0	0	0	0
DepthwiseConv2D, 3 × 3	30 × 40 × 16	30 × 40 × 16	3×3×16×1 = 144	0	144	0
Batch normalization	30 × 40 × 16	30 × 40 × 16	0	0	2×16 = 32	2×16 = 32
ReLU	30 × 40 × 16	30 × 40 × 16	0	0	0	0
Pointwise Conv2D, 1 × 1	30 × 40 × 16	30 × 40 × 8	1×1×16×8 = 128	0	128	0
Batch normalization	30 × 40 × 8	30 × 40 × 8	0	0	2×8 = 16	2×8 = 16
ReLU	30 × 40 × 8	30 × 40 × 8	0	0	0	0
Average pooling, 6 × 8	30 × 40 × 8	5 × 5 × 8	0	0	0	0
Classification Conv2D, 1 × 1	5 × 5 × 8	5 × 5 × 4	1×1×8×4 = 32	4	32+4 = 36	0
Global average pooling	5 × 5 × 4	4	0	0	0	0
Softmax	4	4	0	0	0	0
Total			**448**	**4**	**532**	**80**

BN: trainable scale + offset; non-trainable moving mean + variance (2 per channel each)

Total parameters = 448 weights + 4 biases + 80 trainable BN + 80 non-trainable BN = 612

Dataset Size and Split

70% training and 30% validation, stratified by class

Class	Total	Training (70%)	Validation (30%)
Full Cable	210,000	147,000	63,000
Left End	210,000	147,000	63,000
Right End	210,000	147,000	63,000
No Cable	210,000	147,000	63,000
Total	**840,000**	**588,000**	**252,000**

- Stratified split keeps the same class proportions in training and validation.

No test split is created by this code.

Different augmented versions of the same original may enter training and validation, potentially making validation results overly optimistic.

Counts assume all 210,000 items in each class folder are readable images.

Image Augmentations by Category

- Library: Albumentations for augmentation; OpenCV for image input/output.
- Purpose: increase training variety and reduce overfitting, improving robustness to lighting, blur, noise and camera position.

Category	Augmentations
Lighting and colour (4)	Brightness / contrast + gamma; hue / saturation / value; tone curve; gamma adjustment
Blur (4)	Gaussian blur; motion blur; median blur; defocus blur
Image quality and noise (6)	CLAHE; sharpening; ISO noise; downscaling; multiplicative noise; posterization
Environment and optics (2)	Fog / haze; optical distortion
Geometry (4)	Crop / pad jitter; mild perspective; mild affine transformation; small rotation

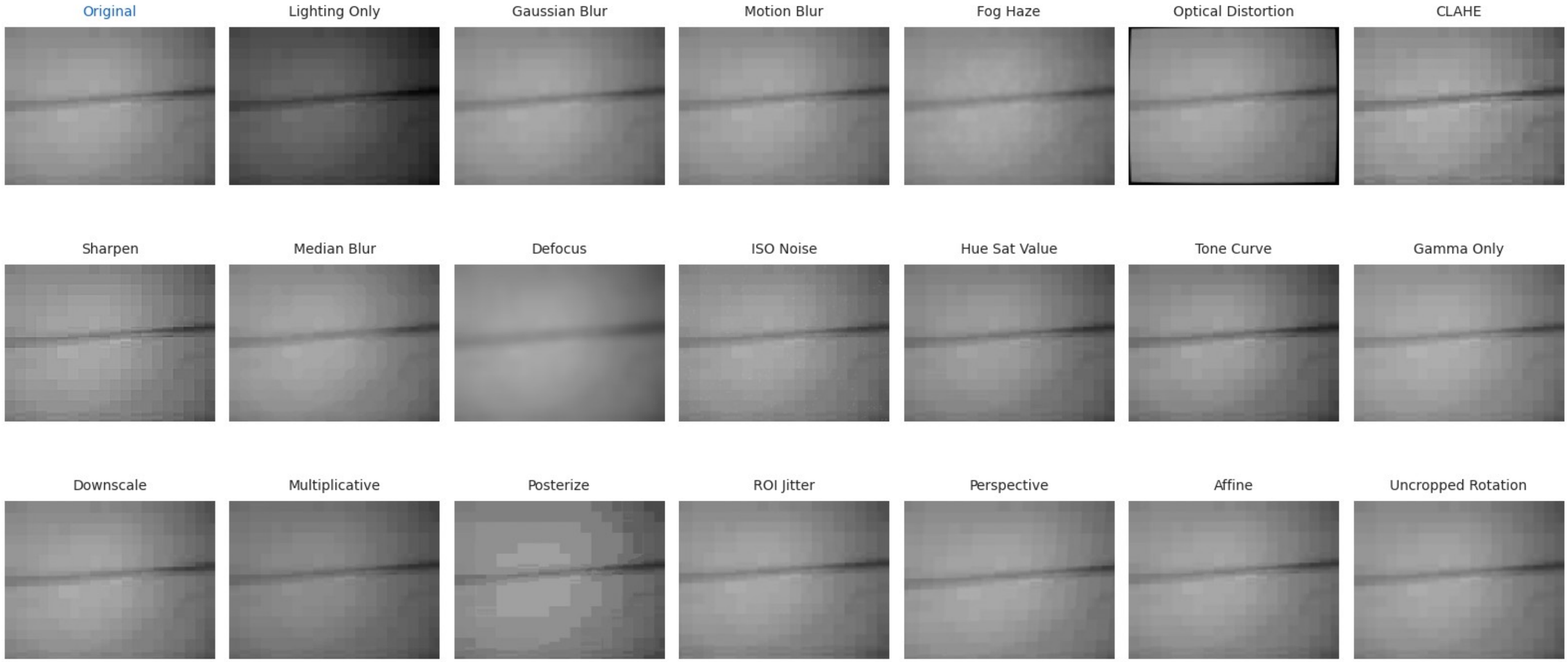
Dataset Size Before and After Augmentation

Each original produces 20 augmented images: 1 original + 20 augmentations = 21 images.

Class	Before augmentation	After augmentation	Training (70%)	Validation (30%)
Full Cable	10,000	210,000	147,000	63,000
Left End	10,000	210,000	147,000	63,000
Right End	10,000	210,000	147,000	63,000
No Cable	10,000	210,000	147,000	63,000
Total	40,000	840,000	588,000	252,000

- Per class: 10,000 × 21 = 210,000 images. Overall: 40,000 × 21 = 840,000 images.
- Stratified split keeps the same class proportions in training and validation.
- No test split. Different versions of the same original may enter both sets, making validation optimistic.

Image Augmentation Examples



Original image and 20 augmented versions

Augmented Testing Dataset

16,731 original images; 351,351 images after augmentation

Class	Before augmentation	After augmentation
Full Cable	4,226	88,746
Left End	4,275	89,775
Right End	4,016	84,336
No Cable	4,214	88,494
Total	**16,731**	**351,351**

Each original receives the same 20 augmentations.

Original + 20 augmented versions = 21 images per original

Original counts = augmented folder counts ÷ 21, assuming all items are images.

Training Loss Function

Sparse categorical cross-entropy

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \ln(p_{i,y_i})$$

N: number of images in the batch
i: image index, from 1 to N
y_i: true class label of image i (0–3)
p_{i,y_i}: predicted probability for its true class

Example: third image is Left End → i = 3, $y_3 = 1$.
$p_{3,1}$ is its predicted probability for Left End.

Class	Label
Full Cable	0
Left End	1
Right End	2
No Cable	3

Why this loss and notation?

- Categorical: one of four classes. Sparse: labels are integers (0–3).
- Cross-entropy penalizes low probability for the correct class.
- Negative sign: $\ln(p) \leq 0$, so $-\ln(p) \geq 0$. Higher correct-class probability gives lower loss.
- Natural log: simpler derivatives. Other log bases only rescale the loss and gradients.

Optimizers: How the Network Learns

An optimizer updates trainable weights and biases using gradients of the loss.

Why needed: the loss measures error. Backpropagation computes gradients. The optimizer chooses the updates.

Common optimizer	Main idea
SGD (Stochastic Gradient Descent)	Updates parameters using the current batch gradient.
SGD with momentum / Nesterov	Uses past gradients to guide updates and reduce oscillation.
Adagrad	Adapts each parameter’s step using accumulated squared gradients.
Adadelta	Uses recent gradient and update history to adapt step sizes.
RMSprop	Scales updates using a moving average of squared gradients.
Adam (Adaptive Moment Estimation)	**Combines momentum with adaptive step sizes for each parameter.**
AdamW	Adam with decoupled weight decay to control weight growth.
Adamax / Nadam	Adam variants using the infinity norm / Nesterov momentum.

- **This CNN uses Adam with an initial learning rate of 0.0003.**
- Optimizers operate during training. FPGA inference uses the trained parameters without updating them.

Adam: Momentum and Adaptive Step Sizes

- Gradient history means information from gradients computed during previous training updates.
- Adam keeps moving averages of gradients (direction) and squared gradients (step scaling), rather than storing every previous gradient.

Aspect	Momentum	Adaptive step sizes
What it tracks	A moving average of gradients.	A moving average of squared gradients.
What it does	Smooths the update direction using current and previous gradients.	Scales the update separately for each weight or bias.
Why it helps	Reduces back-and-forth oscillation and builds progress in consistent directions.	Limits updates where gradients are large and balances parameters with different scales.
Simple analogy	Remembers which way it has been moving.	Adjusts how large a step each parameter takes.
Adam setting	$\beta_1 = 0.9$ controls gradient memory. Smaller β_1: faster direction changes, less smoothing.	$\beta_2 = 0.999$ controls squared-gradient memory. Smaller β_2: faster step scaling, more fluctuation.

Together in Adam

- Momentum provides a smoothed direction. Adaptive scaling adjusts the size of each update.
- The learning rate (initially 0.0003 in this CNN) sets the overall update scale.

Adam Optimizer and Learning-Rate Settings

- An optimizer updates weights to reduce loss. Adam (Adaptive Moment Estimation) combines momentum with adaptive updates for each parameter, often helping training converge efficiently.

Setting	Value	Significance
Optimizer	Adam	Adapts parameter updates using gradient history
Initial learning rate	0.0003	Controls the overall scale of weight updates
β_1	0.9 (default)	Smooths gradients, providing momentum
β_2	0.999 (default)	Smooths squared gradients to adapt individual update sizes
ε	1e−7 (default)	Stabilizes division when squared-gradient estimates are small
AMSGrad	False (default)	Uses standard Adam rather than the AMSGrad variant
Gradient clipping	None	Applies no explicit limit to gradient magnitudes
Learning-rate monitor	Validation loss	Determines when learning-rate reduction is needed
Reduction factor	0.5	Halves the learning rate when improvement stalls
Reduction patience	2 epochs	Waits two epochs without sufficient validation-loss improvement
Minimum learning rate	1e−6	Sets the lower limit for learning-rate reduction

The shaded final four rows describe the ReduceLROnPlateau callback.

Only the initial learning rate is explicitly set in Adam; other optimizer settings use Keras defaults.

How Adam Optimizes Model Parameters

Adam means Adaptive Moment Estimation

1. Compute gradients

Backpropagation measures how each parameter affects loss.

2. Estimate gradient history

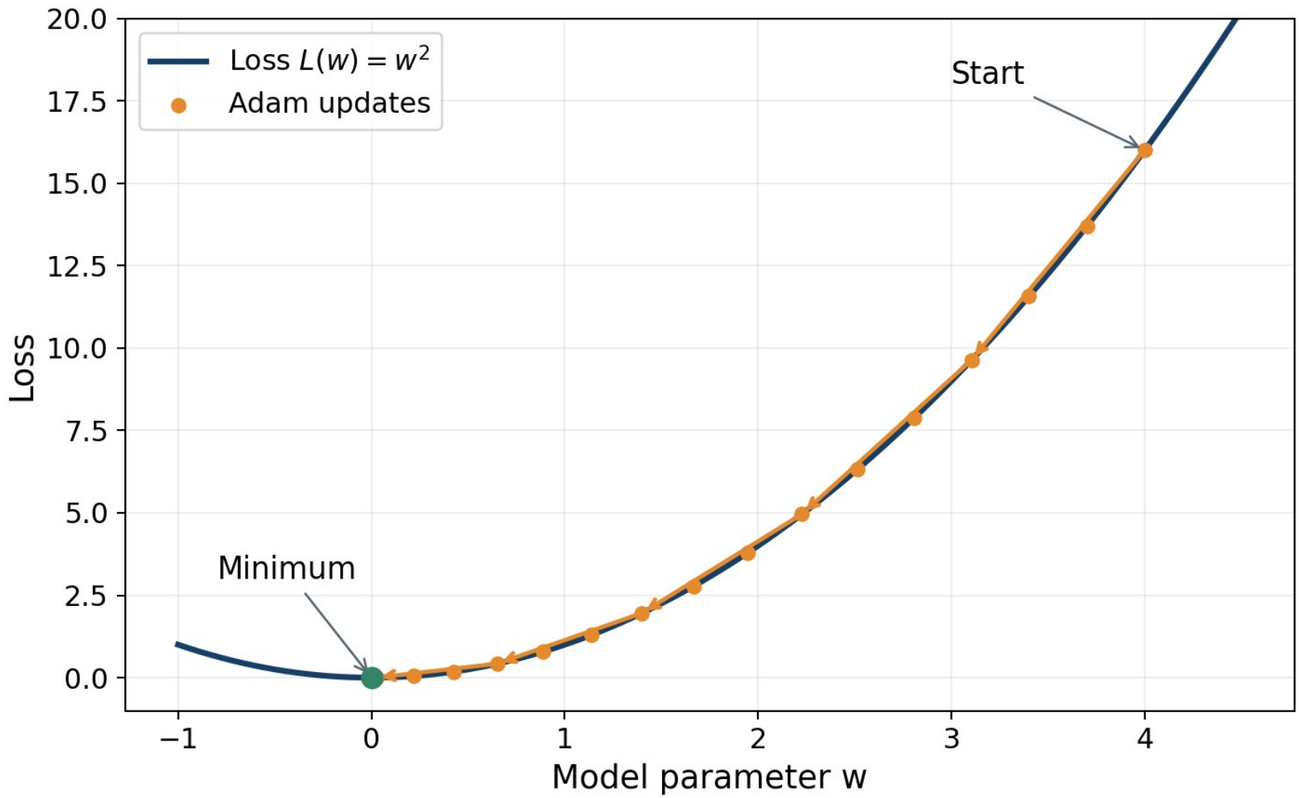
Track moving averages of gradients (momentum) and squared gradients. Correct their initial bias.

3. Update each parameter

Use momentum for direction and the squared-gradient estimate to adapt the update size.

Repeat across training batches

The learning rate controls the overall update scale.



Illustrative example: one parameter, $L(w) = w^2$.
Orange points show 15 Adam updates, learning rate 0.3.

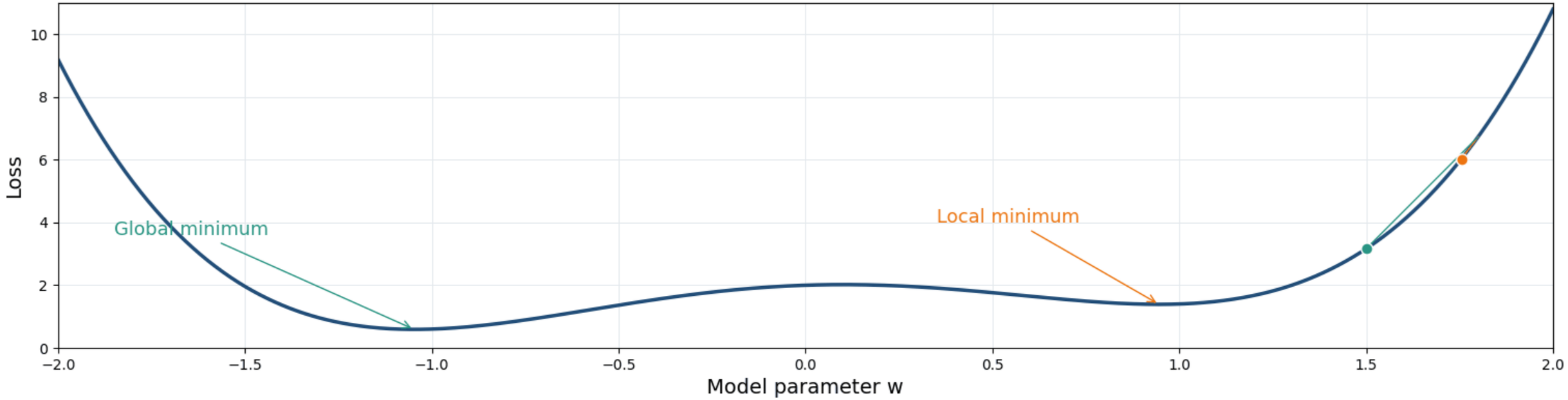
Adam: From Gradient to Weight Update

$$m_t = 0.9m_{t-1} + 0.1g_t$$
$$v_t = 0.999v_{t-1} + 0.001g_t^2$$
$$\hat{m}_t = m_t / (1 - 0.9^t)$$
$$\hat{v}_t = v_t / (1 - 0.999^t)$$
$$\Delta w_t = -\alpha\, \hat{m}_t / (\sqrt{\hat{v}_t} + 10^{-7})$$
$$w_t = w_{t-1} + \Delta w_t$$

Live values — update 1/100

Value	LR 0.045	LR 0.3
g_t	+16.5280	+16.5280
m_t	+1.6528	+1.6528
v_t	+0.2732	+0.2732
$\hat{m}_t$	+16.5280	+16.5280
$\hat{v}_t$	+273.1748	+273.1748
Δw_t	**-0.0450**	**-0.3000**
w_t	+1.7550	+1.5000
Loss	+6.0285	+3.1625

Gradient: g_t Momentum: m_t Squared-gradient history: v_t
Bias correction: $\hat{m}_t$, $\hat{v}_t$ Parameter update: Δw_t, w_t
Learning rate α: sets update size Epsilon: stable division



Past momentum and a larger learning rate can carry updates across a local basin; escape is not guaranteed.

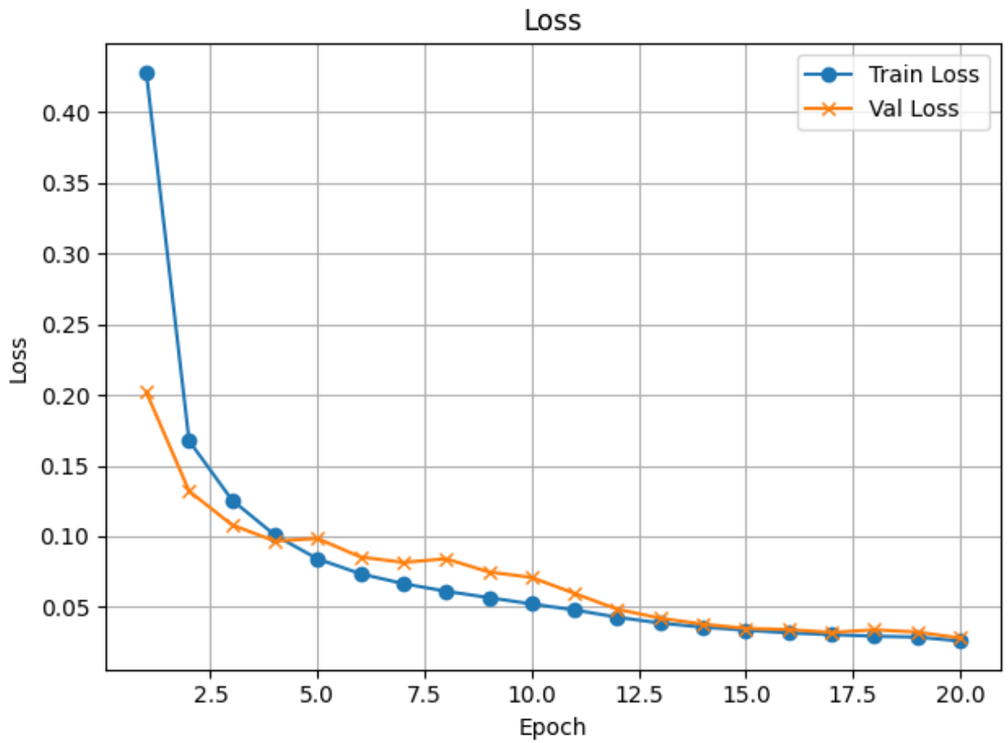
Illustrative loss: $L(w) = (w^2 - 1)^2 + 0.4w + 1$; start $w = 1.8$; $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-7}$.
g_t is evaluated before the update; w_t and loss are shown after it. At $t = 0$, the moment histories start at zero.

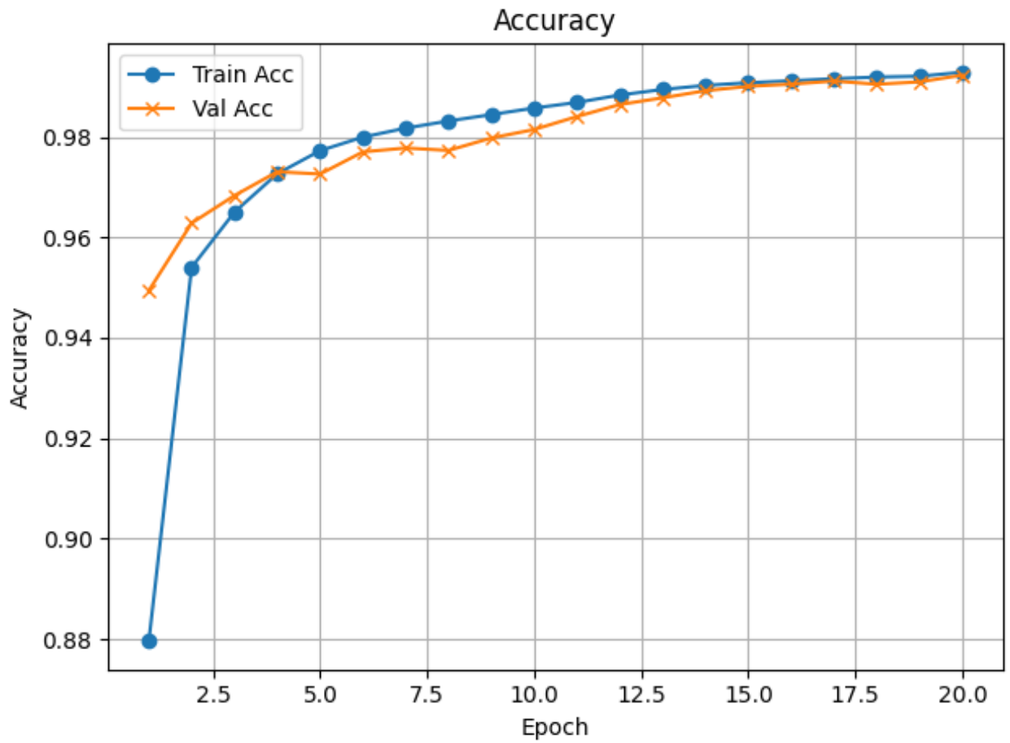
Training and Validation Plots

Plot	What it shows	Why it matters
Loss vs. epoch	Training and validation cross-entropy each epoch	Tracks convergence and helps reveal overfitting
Accuracy vs. epoch	Fraction of correct training and validation predictions	Shows classification progress and the generalization gap
Confusion matrix (counts)	Validation prediction counts for each true/predicted class	Identifies which classes the model confuses
Confusion matrix (normalized)	Fractions within each true class, with rows summing to 1	Compares classes despite different sample counts

Confusion matrices use validation data. No separate test results appear here.

Training and Validation Performance





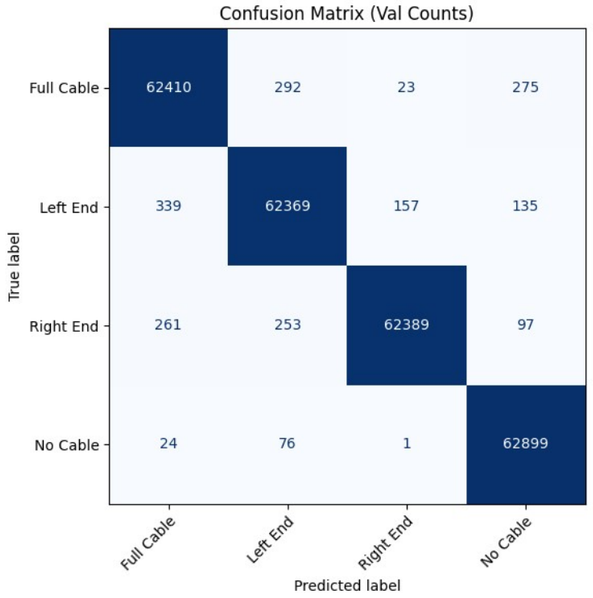
Why training and validation differ

• Early epochs: Training results include mistakes made while the model is still learning. Validation tests the improved model at the end of the epoch, so it can look better.

• Later epochs: The model has learned the training images more closely, so training can perform slightly better than validation.

The curves alone cannot determine each cause's contribution. Both curves converge with a small final gap.

Validation Confusion Matrices

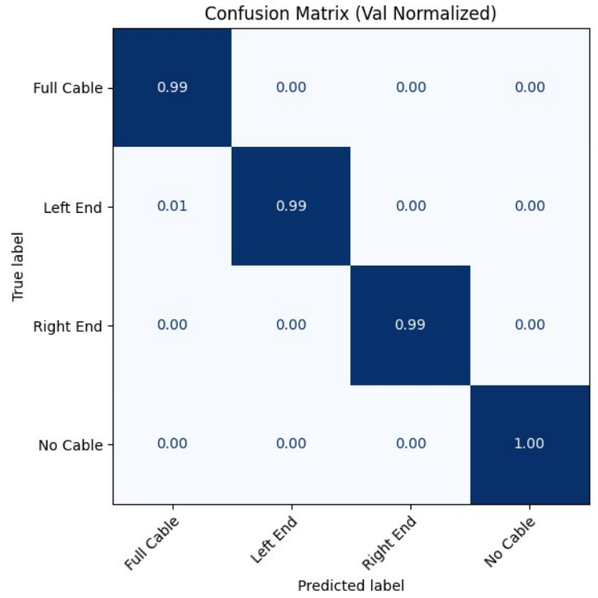


Row normalization

Divide each cell by its row total (63,000 here).

Example: 62,410 ÷ 63,000 = 0.9906 ≈ 99.06%

Shows proportions, making class performance easier to compare.



Information	Computed result	How to read it / why it matters
Correct predictions	250,067 of 252,000	Sum of diagonal values; successful classifications
Misclassifications	1,933 (0.767%)	Sum outside the diagonal; total errors
Common class confusions	Left End → Full Cable: 339; Full Cable → Left End: 292	Largest off-diagonal counts; main mistakes
Recall per class	FC 99.063% · LE 98.998% · RE 99.030% · NC 99.840%	Diagonal ÷ row total; class detection rate
Precision per class	FC 99.010% · LE 99.014% · RE 99.711% · NC 99.200%	Diagonal ÷ column total; prediction reliability
Overall accuracy	250,067 ÷ 252,000 = 99.233%	Fraction of all predictions that are correct
Class support	63,000 per class; 252,000 total	Row totals; validation examples for each actual class

Rows = actual classes; columns = predictions. FC: Full Cable, LE: Left End, RE: Right End, NC: No Cable.

C-Style Weights for FPGA Inference

“C-style” means C/C++ array syntax: braces and commas organize the trained values.

Example: float conv1_weights[9][16] = {{ /* 16 values */ }, /* 9 rows */};

Exported information	What the code does	Why it matters
Convolution weights	Exports conv1, depthwise, pointwise and classifier kernels	Supplies the learned filters for each layer
Batch normalization	Folds BN into conv1, depthwise and pointwise weights and biases	Removes separate BN calculations during inference
Classifier parameters	Exports the 1×1 classifier weights and biases directly	Preserves class scores; no BN is applied to logits
C/C++ arrays	Stores numeric parameters in a form HLS code can include	Uses the trained model without a Python runtime
Array layout	Inference indexing must match the exported kernel order	Prevents incorrect filter and channel mapping
Numeric precision	C-style export is a format; quantization is a separate choice	Allows a deliberate accuracy and hardware-cost tradeoff

C-style describes the representation. BN-folded describes how the values were calculated.

A .txt file stores the array text. HLS code can use it after adding the matching C/C++ declaration.

Exported Parameters and Float32 Storage

Each float32 value uses 32 bits = 4 bytes. Storage = number of values × 4 bytes.

Exported parameter	Value-count calculation	Float32 storage calculation
Conv1 folded weights	3 × 3 × 1 × 16 = 144	144 × 4 = 576 B
Conv1 folded biases	16	16 × 4 = 64 B
Depthwise folded weights	3 × 3 × 16 = 144	144 × 4 = 576 B
Depthwise folded biases	16	16 × 4 = 64 B
Pointwise folded weights	1 × 1 × 16 × 8 = 128	128 × 4 = 512 B
Pointwise folded biases	8	8 × 4 = 32 B
Classifier weights	1 × 1 × 8 × 4 = 32	32 × 4 = 128 B
Classifier biases	4	4 × 4 = 16 B
TOTAL AFTER FOLDING	**448 weights + 44 biases = 492**	**492 × 4 = 1,968 B**

BN values: (16 + 16 + 8) channels × 4 values per channel = 160.

Each BN channel stores scale γ, offset β, moving mean μ and variance σ^2 (80 trainable + 80 statistics).

After folding: 612 − 160 separate BN values + 40 new folded biases = 492 values.

Float32 storage excludes text formatting. The exported .txt files occupy more space on disk.

HLS Latency and Loop Scheduling

Reported timing uses a 10 ns clock period (100 MHz). Times below are in microseconds.

Module / nested loop	Cycles	Time (µs)	Iter. latency	Target II	Achieved II	Function interval	Trip count	Pipelined	Issue
ds_cnn	**344,701**	**3,447.01**	**—**	**—**	**—**	**344,702**	**—**	**No**	**—**
Conv1 + ReLU + MaxPool	**192,013**	**1,920.13**	**—**	**—**	**—**	**192,013**	**—**	**No**	**II violation**
↳ Conv1 loop (26/27/28)	192,011	1,920.11	17	1	10	—	19,200	Yes	**II violation**
Average pooling	**4,808**	**48.08**	**—**	**—**	**—**	**4,808**	**—**	**No**	**II violation**
↳ AvgPool loop (136/137/138)	4,806	48.06	31	1	24	—	200	Yes	**II violation**
Depthwise + Pointwise	**142,801**	**1,428.01**	**—**	**—**	**—**	**142,801**	**—**	**No**	**II violation**
↳ DW/PW loop (78/79)	142,800	1,428.00	119	—	—	—	1,200	No	—
↳ Depthwise loop (85)	87	0.87	13	1	5	—	16	Yes	**II violation**
↳ Pointwise loop (111)	27	0.27	21	1	1	—	8	Yes	—
GAP + Classifier + Argmax	**262**	**2.62**	**—**	**—**	**—**	**262**	**—**	**No**	**II violation**
↳ GAP initialization (165)	8	0.08	1	—	1	—	8	Yes	—
↳ GAP sum (168/169/170)	203	2.03	4	1	1	—	200	Yes	—
↳ GAP scaling (178)	24	0.24	4	—	3	—	8	Yes	**II violation**
↳ Classifier (185)	14	0.14	11	1	1	—	4	Yes	—
↳ Argmax (198)	3	0.03	1	—	1	—	3	Yes	—
Image load (231)	4,801	48.01	3	1	1	—	4,800	Yes	—

- One image takes about 3.45 ms. Conv1 and depthwise–pointwise account for about 97% of latency.
- Indentation shows loop nesting. Depthwise requests II=1 but achieves II=5; the outer DW/PW loop has no II warning.
- II=1 may be limited by memory ports or dependencies between calculations. The warning details identify the exact cause.
- Numbers in parentheses are source-code line numbers: (78/79) identifies the nested loops at lines 78 and 79.

CNN Weight Headers: What Each File Does

One header specifies the arrays. The other fills them with the trained weights and biases.

File	Role
ds_cnn_weights.h	Specifies the number format (data_t) and network sizes. Lists each weight/bias array name and shape. extern tells the compiler that its values are defined elsewhere.
ds_cnn_weights_data.h	Creates the arrays and fills them with trained values. Uses #include to insert the exported C-style .txt values. Converts those values to the fixed-point type data_t.

- The C-style numbers are in weights_cstyle_folded/*.txt. ds_cnn_weights_data.h includes them in the model.

- In the design, the visible weight modules use LUT-based logic, while image and feature buffers use BRAM.

HLS Resource Estimates

Estimated hardware resources for the complete accelerator and its main functions.

Resources by module (HLS estimates)

Module	BRAM18	DSP	FF	LUT	URAM
ds_cnn: complete accelerator	57	71	6,570	12,474	0
Conv1 + ReLU + MaxPool	0	36	1,506	4,805	0
Average pooling	0	1	977	3,605	0
Depthwise + Pointwise	0	25	1,806	1,447	0
GAP + Classifier + Argmax	0	9	1,343	1,257	0

Complete accelerator compared with Zynq-7010 capacity

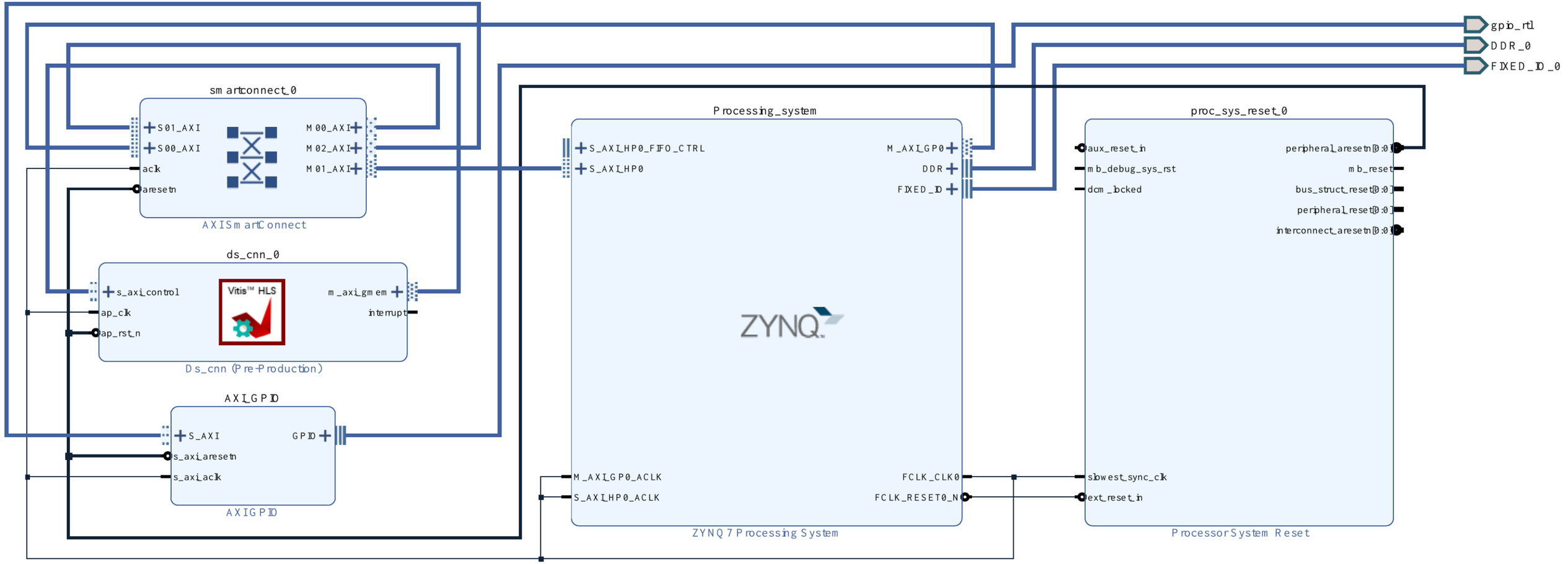
Resource	HLS estimate	Zynq-7010 available	Estimated utilization
BRAM (18 Kb)	57	120	47.5%
DSP	71	80	88.8%
FF	6,570	35,200	18.7%
LUT	12,474	17,600	70.9%
URAM	0	0	Not available

- Module rows show where resources are used; the top total also includes memories and interface/control logic.
- BRAM: 120 × 18 Kb = 60 × 36 Kb blocks. These are estimates; confirm final utilization in Vivado.

CNN Accelerator: Vivado Block Design

External ports

DDR_0: external memory gpio_rtl: general-purpose I/O FIXED_IO_0: PS pins (MIO, clock, reset)



Block Design: Module Purposes

Module name	Purpose
Processing_system ZYNQ7 Processing System	Runs software on the ARM processor, controls the CNN accelerator and provides access to external DDR memory.
ds_cnn_0 CNN accelerator	Implements the trained CNN in FPGA logic. Reads the input image, computes class scores and returns the predicted class.
smartconnect_0 AXI SmartConnect	Routes AXI transactions between the processor, CNN accelerator, GPIO and the processing-system memory interface.
AXI_GPIO AXI GPIO	Provides software-controlled digital inputs/outputs through AXI registers and the external gpio_rtl port.
proc_sys_reset_0 Processor System Reset	Generates synchronized reset signals for the accelerator and AXI peripherals using the supplied clock and reset request.

• The processor controls execution; the FPGA accelerator performs CNN inference; AXI connects the hardware modules.

Block Design: Modules and Connections

Connection endpoint A (pin / interface)	Connection endpoint B (pin / interface)	Purpose
Processing_system (M_AXI_GP0)	smartconnect_0 (S01_AXI)	CPU access to accelerator/GPIO control registers
ds_cnn_0 (m_axi_gmem)	smartconnect_0 (S00_AXI)	CNN initiates image reads from memory
smartconnect_0 (M00_AXI)	ds_cnn_0 (s_axi_control)	Configure/start CNN and read its results
smartconnect_0 (M01_AXI)	Processing_system (S_AXI_HP0)	High-performance access to PS DDR memory
smartconnect_0 (M02_AXI)	AXI_GPIO (S_AXI)	CPU reads/writes GPIO registers
Processing_system (DDR)	External port (DDR_0)	Connect the PS to external DDR memory
Processing_system (FIXED_IO)	External port (FIXED_IO_0)	Dedicated PS I/O, clock and reset pins
AXI_GPIO (GPIO)	External port (gpio_rtl)	General-purpose digital input/output
Processing_system (FCLK_CLK0)	proc_sys_reset_0 (slowest_sync_clk)	Clock used to synchronize reset release
Processing_system (FCLK_RESET0_N)	proc_sys_reset_0 (ext_reset_in)	Provide PS reset request to reset controller
Processing_system (FCLK_CLK0)	ds_cnn_0 (ap_clk); AXI_GPIO (s_axi_aclk); smartconnect_0 (aclk); PS (AXI clocks)	Clock the AXI interfaces and accelerator
proc_sys_reset_0 (peripheral_aresetn)	ds_cnn_0 (ap_rst_n); AXI_GPIO (s_axi_aresetn); smartconnect_0 (aresetn)	Distribute active-low reset

• Each row identifies connected endpoints. AXI carries requests and responses; GPIO direction depends on its configuration.

Address Map: Memory and Control Registers

Assigned address ranges shown in the Vivado Address Editor

Accessed by	Target	Base address	High address	Size
CNN accelerator (m_axi_gmem)	PS memory via S_AXI_HP0 HP0_DDR_LOWOCM	0x00000000	0x1FFFFFFF	512 MiB
ARM processor	AXI GPIO registers	0x40000000	0x4000FFFF	64 KiB
ARM processor	CNN control/result registers	0x40010000	0x4001FFFF	64 KiB

- The first range provides access to DDR DRAM and the low-address on-chip memory (OCM) mapping.
- GPIO and CNN ranges are register address spaces. A 64 KiB allocation does not mean 64 KiB of stored data.
- Register address = peripheral base address + register offset (from the generated driver header).

  Example: CNN control register = 0x40010000 + 0x00 = 0x40010000.

Role of the Bare-Metal Vitis Application

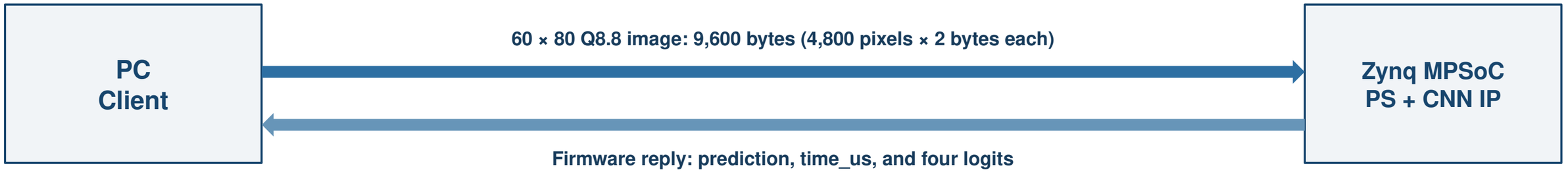
The Zynq PS software connects the PC data stream to the CNN accelerator

01	**RECEIVE THE IMAGE**	Finds the A6 6A frame header and receives 4,800 signed Q8.8 pixels (9,600 payload bytes).
02	**PREPARE DDR INPUT**	Stores the samples in DDR and flushes the PS cache so the PL reads the latest image data.
03	**CONTROL THE CNN IP**	Writes the input pointer through AXI-Lite, starts the accelerator, and waits for ap_done.
04	**RETURN THE RESULTS**	Reads the prediction and four logits, measures elapsed time, and sends the 15-byte reply.

Bare-metal C gives the PS direct control of UART, DDR, and the CNN registers without an operating system.

PC–MPSoC Data Flow

The PS receives the image, controls the CNN accelerator, and returns its results



CNN register offsets and absolute addresses

Register	Offset	Absolute address	Firmware action
AP_CTRL	0x00	0x40010000	Start; poll ap_done
INPUT_IMG_DATA	0x10 / 0x14	0x40010010 / 0x40010014	Write 64-bit DDR pointer
PRED_OUT_DATA	0x1C	0x4001001C	Read predicted class
LOGITS_OUT	0x30 / 0x34	0x40010030 / 0x40010034	Read four 16-bit logits

Vivado address map

Access path	Base	High	Size
CNN m_axi → HP0	0x00000000	0x1FFFFFFF	512 MiB
AXI GPIO	0x40000000	0x4000FFFF	64 KiB
CNN AXI-Lite	0x40010000	0x4001FFFF	64 KiB

- The PS flushes the data cache so the CNN reads the latest image values from DDR through m_axi.
- The PS timer measures start to ap_done; the GPIO pin marks the accelerator run interval.
- Firmware sends 15 reply bytes: prediction, time_us, and four signed Q8.8 logits.
- Qm.n means m integer bits and n fractional bits; Q8.8 has 8 of each (including the sign bit).

UART Frame Byte Layout

PC sends the image frame; firmware returns prediction, time, and logits

PC → MPSoC | Image frame: 9,604 bytes total

Byte 0 0xA6 Start byte 0	Byte 1 0x6A Start byte 1	Bytes 2–3 80 25 Length = 9,600 (uint16 LE)	Bytes 4–9,603	Payload: 9,600 bytes 4,800 signed int16 Q8.8 pixels, little-endian

The length is the payload size. This input frame has no separate end marker or checksum.

MPSoC → PC | Firmware reply: 15 bytes

Byte index	Field	Value / encoding
0	Start marker	0xA5
1	Predicted class	pred (uint8)
2–5	Inference time	time_us (uint32, least-significant byte first)
6–7	Logit 0	signed int16, little-endian
8–9	Logit 1	signed int16, little-endian
10–11	Logit 2	signed int16, little-endian
12–13	Logit 3	signed int16, little-endian
14	End marker	0x5A

Byte order is little-endian for time_us and each int16 logit. The reply has start/end markers but no checksum.

Student Exercises: CNN on FPGA

Take the trained CNN from HLS verification to on-board image classification.

01 Verify the design in Vitis HLS

Write a C/C++ testbench and compare accelerator outputs with the software reference. Check HLS estimates for LUT, FF, BRAM, DSP use, and latency.

02 Export RTL and integrate in Vivado

Generate synthesizable RTL and package the HLS design as IP. Add it to the block design, then run synthesis and implementation.

03 Generate the bitstream and program the FPGA

Build the bitstream in Vivado and load it onto the FPGA board.

04 Run test images with a bare-metal Vitis app

Send each test image to the accelerator, read its predicted class, and compare the result with the expected label.